

MATERI PERTEMUAN 3

1. TUJUAN

CPMK: Mahasiswa mengetahui dan memahami konsep DOM (Document Object Model) dan event handling dalam JavaScript.

Sub-CPMK:

- a. Mahasiswa mampu memahami struktur dan cara kerja DOM dalam manipulasi elemen HTML.
- b. Mahasiswa mampu mengimplementasikan event handling untuk menangani berbagai jenis interaksi pengguna menggunakan JavaScript.

2. DURASI WAKTU

1 pertemuan x 3 jam

3. DASAR TEORI

JavaScript

4. PERCOBAAN

A. Membuat Formulir Interaktif

1. Buatlah sebuah halaman yang akan menampilkan halaman seperti dibawah ini.

Formulir Interaktif - P5_XXX

Masukkan Nama:

2. Buatlah projek baru. Klik **File > New File > HTML**, lalu beri nama **P5_XXX**. Tambahkan struktur dasar HTML berikut:

```
1  <!DOCTYPE html>
2  <html lang="en">
3
4  <head>
5      <meta charset="UTF-8">
6      <meta name="viewport" content="width=device-width, initial-scale=1.0">
7      <title>Formulir Interaktif</title>
8      <style>
9      </style>
10 </head>
11
12 <body>
13     <h1>Formulir Interaktif - P5_XXX</h1>
14     <form id="myForm">
15         <label for="userInput">Masukkan Nama:</label>
16         <input type="text" id="userInput" placeholder="Masukkan nama Anda">
17         <button type="submit">Kirim</button>
18     </form>
19     <p id="message"></p>
20 </body>
21
22 </html>
```

3. Selanjutnya tambahkan tag `<script>` yang digunakan untuk menangani pengiriman formulir secara interaktif. Pada kode di bawah ini terdapat event listener yang ditambahkan pada formulir untuk menangani event 'submit' ketika button Kirim diklik.

```
21 <script>
22
23     document.addEventListener('DOMContentLoaded', () => {
24         const form = document.getElementById('myForm');
25         const message = document.getElementById('message');
26
27         form.addEventListener('submit', (event) => {
28             event.preventDefault(); // Mencegah formulir dari pengiriman default
29
30             const userInput = document.getElementById('userInput').value;
31             if (userInput.trim()) {
32                 message.textContent = `Terima kasih, ${userInput}! Formulir telah dikirim.`;
33             }
34         });
35     });
36
37 </script>
38 </body>
```

4. Coba jalankan P5_XXX.html untuk melihat hasil dari form interaktif yang telah dibuat, berikut merupakan hasil dari form interaktifnya.

Formulir Interaktif - P5_XXX

Masukkan Nama:

Terima kasih, Roni! Formulir telah dikirim.

5. Selanjutnya coba lakukan sedikit modifikasi pada kode untuk menambahkan validasi hasil yang ditampilkan apabila formulir masih kosong. Tambahkan else condition seperti yang terdapat pada baris kode 33 hingga baris kode 35.

```
30 const userInput = document.getElementById('userInput').value;
31 if (userInput.trim()) {
32   message.textContent = `Terima kasih, ${userInput}! Formulir telah dikirim.`;
33 } else {
34   message.textContent = 'Silakan masukkan nama Anda.';
35 }
```

6. Lalu refresh dan coba jalankan kembali tanpa melakukan inputan dan langsung klik button “Kirim”. Berikut merupakan hasil dari validasi apabila inputan kosong.

Formulir Interaktif - P5_XXX

Masukkan Nama:

Silakan masukkan nama Anda.

B. Membuat To-Do List dengan Interaksi

1. Buatlah sebuah halaman yang akan menampilkan halaman seperti dibawah ini.

Daftar Tugas - P6_XXX

- ☒ ~~Pemrograman 2~~
- ☐ Basis Data

2. Buka kembali proyek sebelumnya **P3_XXX**, salin seluruh kodenya kemudian buatlah proyek baru. Klik **File > New File > HTML**, lalu beri nama **P6_XXX**. Tempel source code yang telah disalin ke dalam file **P6_XXX**, lalu ubah kode berikut.

```
<h1>Daftar Tugas - P6_XXX</h1>
```

3. Selanjutnya modifikasi tag `<head>` dengan menambahkan tag `<style>` seperti berikut.

```
3  <head>
4      <meta charset="UTF-8">
5      <meta http-equiv="X-UA-Compatible" content="IE=edge">
6      <meta name="viewport" content="width=device-width, initial-scale=1.0">
7      <title>Daftar Tugas</title>
8      <style>
9          .completed {
10              text-decoration: line-through;
11              color: gray;
12          }
13          button {
14              margin-left: 10px;
15          }
16      </style>
17  </head>
```

4. Lakukan modifikasi pada fungsi `tampilkanTugas()`, modifikasi pada kode yang dilakukan bertujuan untuk menambahkan checkbox yang nantinya sebagai penanda apakah tugas sudah selesai atau belum.

```

50 // Fungsi untuk menampilkan daftar tugas
51 function tampilkanTugas() {
52     const list = document.getElementById('listTugas');
53     list.innerHTML = ''; // Membersihkan daftar sebelum ditampilkan
54
55     if (daftarTugas.length === 0) {
56         list.innerHTML = '<li>Tidak ada tugas.</li>'; // Menampilkan pesan jika tidak ada tugas
57     } else {
58         daftarTugas.forEach((tugas, index) => {
59             // Membuat elemen list untuk setiap tugas
60             const li = document.createElement('li');
61             const checkbox = document.createElement('input');
62             checkbox.type = 'checkbox';
63             checkbox.checked = tugas.completed; // Menentukan apakah checkbox dicentang atau tidak
64             checkbox.onchange = () => tandaiSelesai(index); // Menambahkan fungsi tandai selesai pada checkbox
65
66             const span = document.createElement('span');
67             span.textContent = tugas.text; // Menampilkan teks tugas
68             if (tugas.completed) {
69                 span.classList.add('completed'); // Menandai tugas selesai
70             }
71
72             const btnHapus = document.createElement('button');
73             btnHapus.textContent = 'Hapus';
74             btnHapus.style.marginLeft = '10px'; // Menambahkan margin kiri langsung pada tombol
75             btnHapus.onclick = () => hapusTugas(index); // Menambahkan fungsi hapus pada tombol
76
77             li.appendChild(checkbox); // Menambahkan checkbox ke elemen li
78             li.appendChild(span); // Menambahkan teks tugas ke elemen li
79             li.appendChild(btnHapus); // Menambahkan tombol hapus ke elemen li
80
81             list.appendChild(li); // Menambahkan tugas ke daftar
82         });
83     }
84
85     // Tampilkan daftar hanya saat tombol ditekan
86     list.style.display = 'block';
87 }

```

5. Coba jalankan P6_XXX.html lalu tambahkan beberapa tugas dan klik “Tampilkan Daftar Tugas”. Berikut merupakan hasil dari modifikasi kode yang dilakukan sebelumnya.

Daftar Tugas - P6_XXX

- ☐ Pemrograman 3
- ☐ Pengantar Informatika

6. Selanjutnya tambahkan fungsi untuk menandai tugas yang diceklis melalui checkbox.

```

89 // Fungsi untuk menandai tugas sebagai selesai atau belum selesai
90 function tandaiSelesai(index) {
91     daftarTugas[index].completed = !daftarTugas[index].completed;
92     tampilkanTugas(); // Memperbarui tampilan setelah perubahan status
93 }

```

7. Coba refresh dan jalankan kembali P6_XXX.html lalu klik pada salah satu checkbox untuk menandai tugas telah selesai.

Daftar Tugas - P6_XXX

- ☒ Pemrograman 3
- ☐ Basis Data

5. LATIHAN

1. Aplikasi Penghitung Jumlah Klik

Buatlah sebuah aplikasi sederhana di halaman web yang memiliki tombol. Setiap kali tombol tersebut diklik, aplikasi akan menghitung dan menampilkan jumlah klik yang telah dilakukan. Berikut merupakan instruksi dari tugas yang diberikan penamaan file T4_XXX:

Instruksi:

- Buat sebuah tombol di halaman web.
- Setiap kali tombol diklik, tampilkan jumlah total klik pada halaman tersebut.

2. Tombol Pengubah Warna Latar Belakang Secara Acak

Buatlah sebuah tombol yang, saat diklik, akan mengubah warna latar belakang halaman secara acak. Berikut merupakan instruksi dari tugas yang diberikan penamaan file T5_XXX:

Instruksi:

- Buat sebuah tombol di halaman web.
- Setiap kali tombol diklik, ubah warna latar belakang halaman ke warna yang dipilih secara acak.